

数据结构

您需要写一种数据结构来维护一些数，其中需要提供以下操作：

1. 插入 x 数
2. 删除 x 数(若有多个相同的数，因只删除一个)
3. 查询 x 数的排名(排名定义为比当前数小的数的个数+1。若有多个相同的数，因输出最小的排名)
4. 查询排名为 x 的数
5. 求 x 的前驱(前驱定义为小于 x ，且最大的数)
6. 求 x 的后继(后继定义为大于 x ，且最小的数)

有序表

根据上述操作，我们不难想到暴力的做法，因为我们需要知道一个数 x 在数列中的位置，已知他的前驱和后继，那么我们维护一个有序表就可以了。即数列是有序的，每次把数插入到对应的位置，我们来思考一下时间复杂度：

- (1) 插入数 x ：因为是有有序表，所以我们可以考虑二分查找，找到位置的时间复杂度为 $O(\log n)$ ，但是每次插入后面的数又要向后挪一位，所以最坏时间复杂度为 $O(n)$
- (2) 删除数 x ：和插入相同。
- (3) 数 x 的排名：二分查找， $O(\log n)$
- (4) 排名为 x 的数：下标查找 $O(1)$
- (5) x 的前驱，二分查找， $O(\log n)$
- (6) x 的后继，同前驱。

有序表

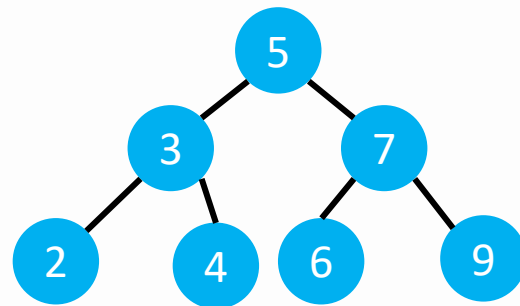
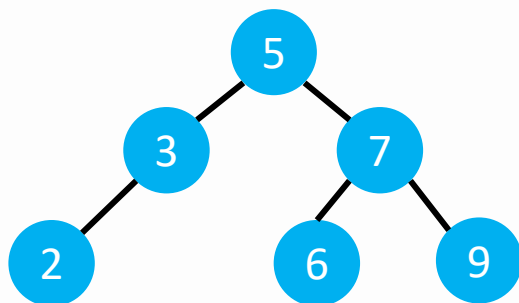
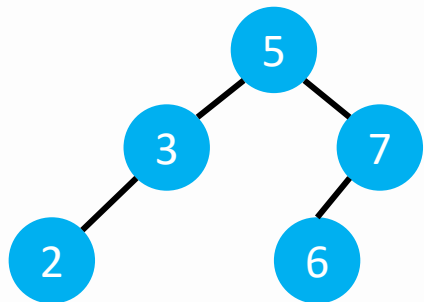
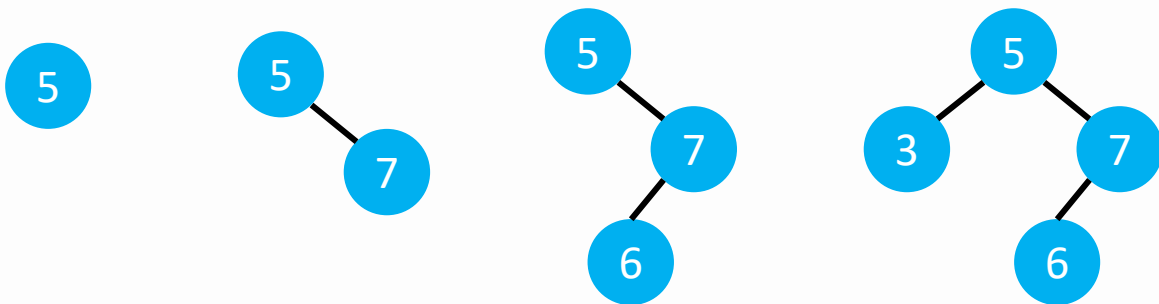
对于竞赛来说， $\log n$ 的算法就已经很优秀了，但是在插入和删除上的低效率就决定了被淘汰的命运。我们再思考一下如何在插入删除上优化，对于顺序表，插入和删除都是 $O(n)$ 的，并且又要可以使用二分的思想，所以我们只能考虑树形结构。

比如堆，堆的插入删除虽然是 $\log n$ 的，但是堆没有办法直接取中间任意元素，只能访问修改队首，也不符合我们的操作。

我们可以考虑来维护一颗有序的树，即该数的左子树一定比根结点小，右子树一定比根结点数大。这样当我们中序遍历这棵树的时候就得到了一个有序表。

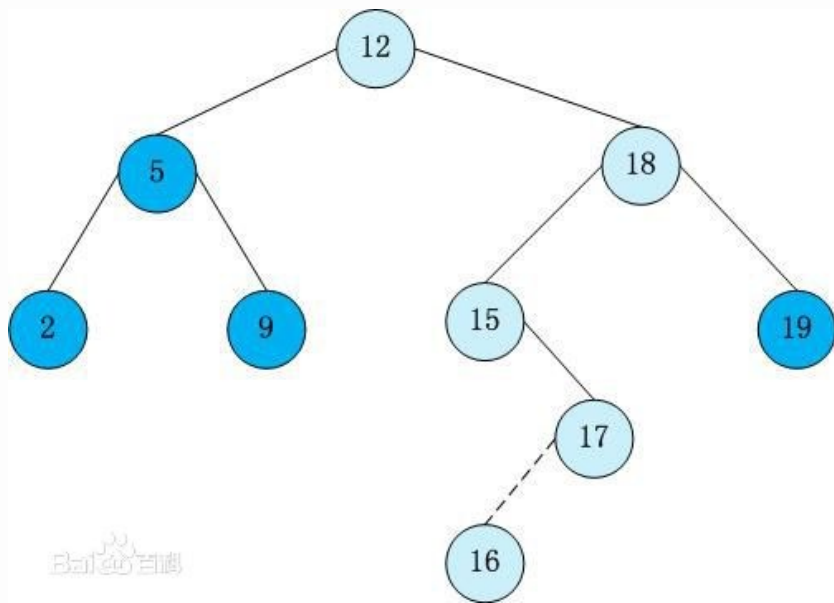
有序树

比如 5 7 6 3 2 9 4，建树的过程为



二叉搜索树

二叉搜索树：又名二叉查找树，二叉排序树。具有以下性质：若他的左子树不为空，那么他的左子树上所有节点的值均小于根结点的值。若他的右子树不为空，那么他的右子树上所有节点的值均大于根结点。同样的，他的左右子树均满足二叉搜索树。



堆：根结点均大于(小于)左右子树的完全二叉树。

应用

因为二叉搜索树的左子树均小于他的根，右子树均大于他的根，所以具有很强的实际应用。

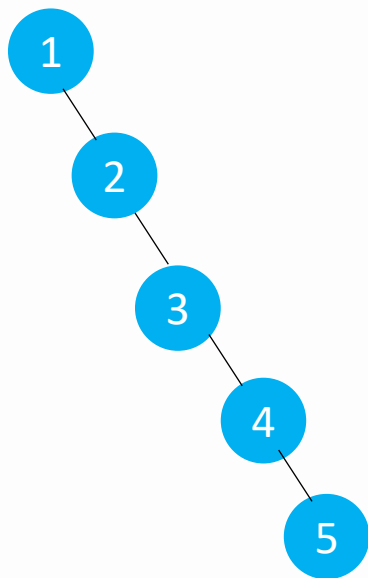
- 1: 排序
- 2: 查找第k大的值。
- 3: 判断一个数是第几大的数。
- 4: 修改并查询

由于它是有顺序的树形结构，我们可以通过二分的办法很快对该树进行搜索。理论上的时间复杂度为 $O(n \log n)$. 但是!!!

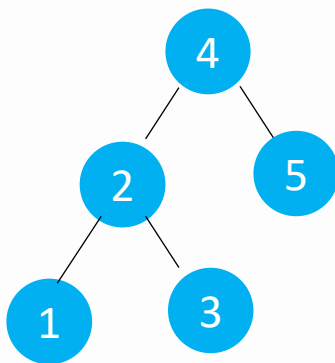
二叉搜索树

我们在实际操作中会发现一个问题——二叉搜索树的建立不是唯一的。他的形态取决于我们建树的方法。如果按照一般情况下，边输入边维护二叉搜索树的办法建树，会存在一定的问题。

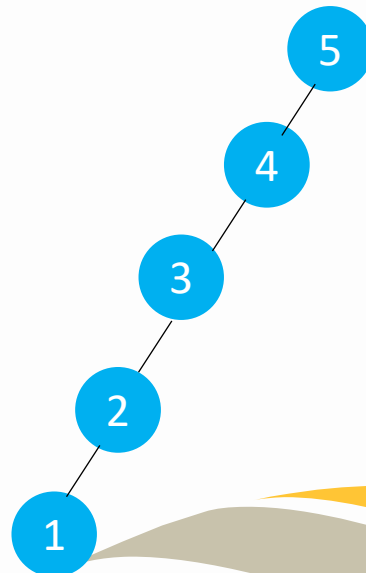
1 2 3 4 5



4 5 2 1 3



5 4 3 2 1



不足之处

从上图可以看出，我们建立的二茬搜索树的形态取决于数据输入的顺序，如果每次输入的数相对于根结点来说比较中间或者使用随机数，生成的二叉搜索树是比较接近完全二叉树的，时间复杂度就比较低，查找的时间复杂度近似为 $O(n \log n)$ 。但是如果输入的数据大部分都是有序的，那么生成的二叉搜索树形态就比较接近一条链，甚至就是一条链。搜索效率肯定就比较第，还不如直接用线性表。所以直接使用二叉搜索树在竞赛中很难得到高分。

通过观察我们会发现，如果一棵二叉搜索树的左右子树的数量越相等，查找效率就越高。即如果一棵二叉搜索树越平衡，查找效率就越高。所以我们要保证二叉搜索树的平衡性。这样的二叉搜索树叫平衡二叉搜索树。简称平衡树。

平衡树

平衡树：它是一棵空树或它的左右两个子树的高度差的绝对值不超过1，并且左右两个子树都是一棵平衡二叉树

由于输入的数据是固定的，并且还有多次删除和添加以及修改操作。所以我们需要想办法来维护该树的平衡性。维护平衡性的方法有很多种。比如AVL树，红黑树，Treap，Splay。我们先学习一种简单的平衡树——Treap。

Treap

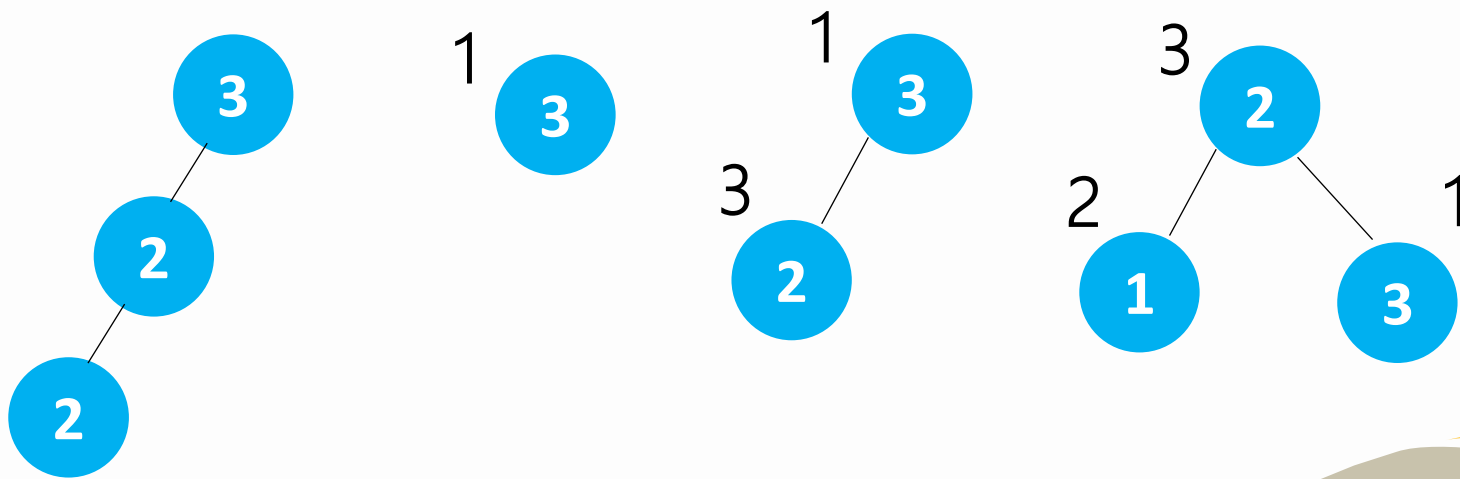
Treap: 实质就是Tree+Heap. 即用二叉堆的性质来维护二叉搜索树的平衡。

Treap树既要满足二叉搜索树的性质(根结点小于左子树, 大于右子树), 还要满足大根堆的性质(根结点大于左右子树)。这样看上去似乎是不可能的。所以我们让每个节点拥有两个值。一个值满足二叉搜索树, 一个值满足二叉堆。这样就完美解决了这个问题。

Treap

比如输入3 2 1

如果直接按照二叉搜索树的方法来建立，就是一条直链，
但是如果我们给每个节点再添加一个值(假设为优先级)，
1 3 2。如果是链的形式明显不满足大根堆。所以要变形
(旋转) 直到满足条件为止(图为大根堆)



确定优先级

再输入的时候，我们需要在给每一个数赋值一个优先级，但是很明显我们是不知道当前这个数在输入的数中的优先级的。前面我们也提到了，如果输入的数据是随机数，结果可以接近 $n\log n$ 。所以我们直接给每次输入的数据随机生成一个优先级就可以了。经过多次试验，效率比较接近 $n\log n$ 。

生成整形随机数：

```
#include<cstdlib>  
x=rand();
```

生成 $[a, b]$ 之间的随机数

```
x=a+rand()%(b+1-a);
```

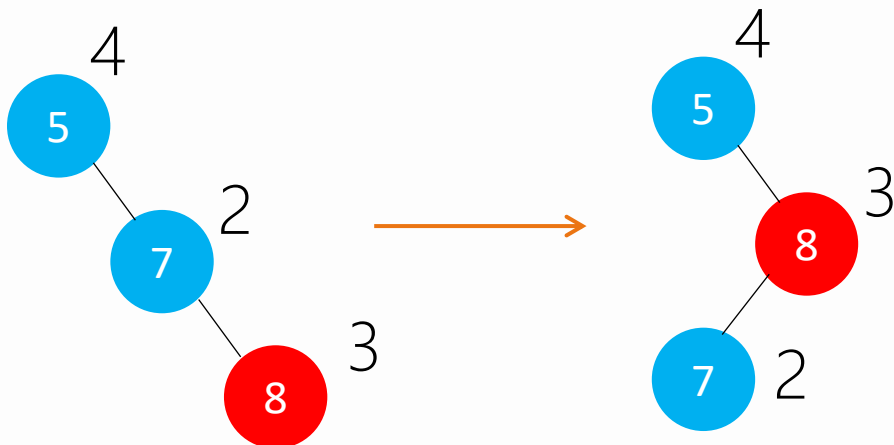
平衡树的维护

对于一个纯粹的二叉搜索树，如果要新建一个节点，我们肯定把这个节点直接放在原二叉搜索树的一个空位置让他仍然满足原性质。（和堆不一样，插入一个数到堆里面需要一直交换父亲和儿子结点）

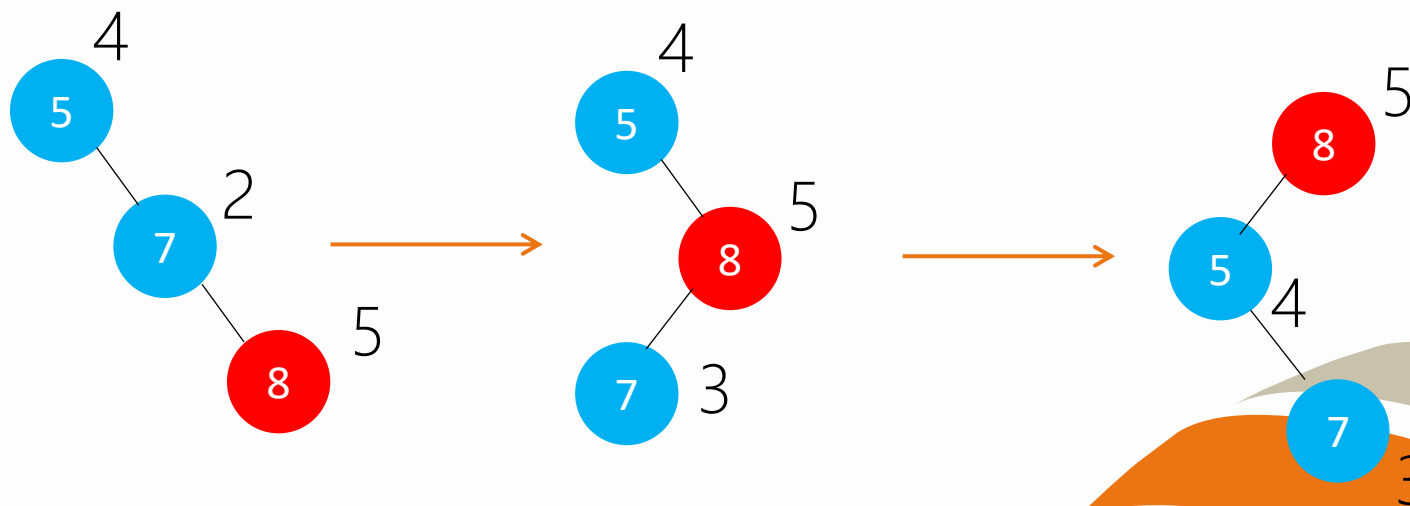
我们可以先根据插入的数的实际大小先把该数插入二叉搜索树中合适的位置并且保持原性质，然后通过某些操作(旋转)让他保持二叉搜索树的性质不变并且满足大根堆的性质，这种操作我们称之为treap树的旋转，旋转分为左旋和右旋。

旋转

我们先考虑新插入的节点在右子树的情况。

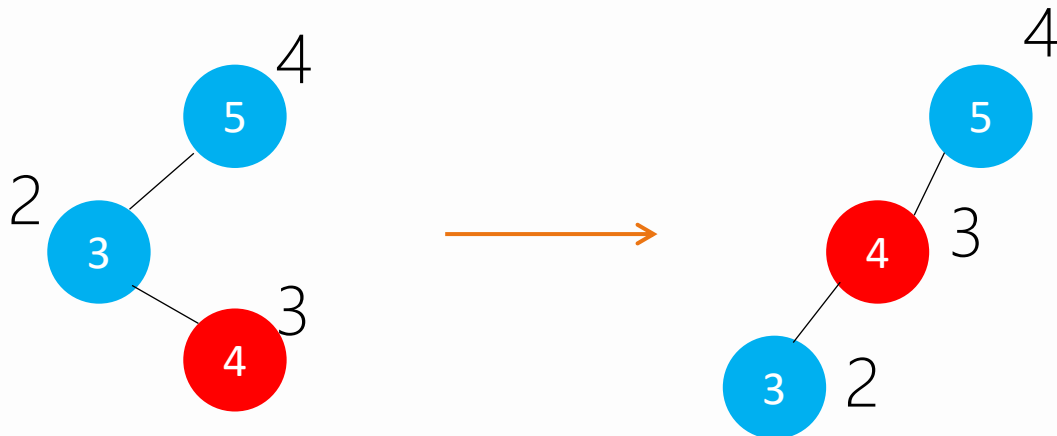


如果右子树插入的节点优先级比父亲节点小，就不需要旋转。



旋转

我们先考虑新插入的节点在右子树的情况。



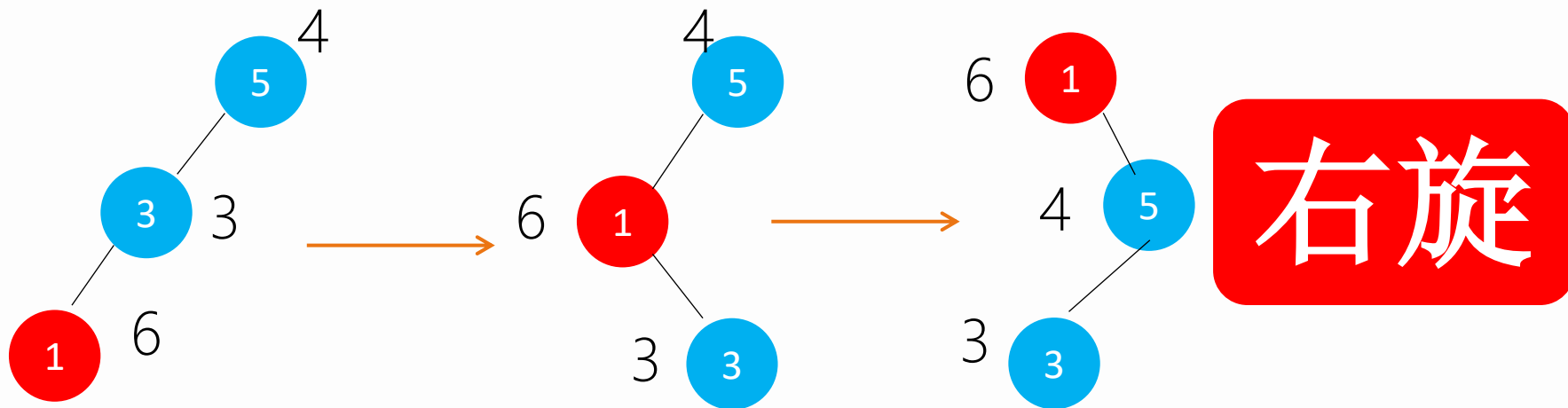
左旋

经过前面的分析，如果是在右子树插入新的节点：

1. 插入节点的优先级 < 根结点的优先级，满足大根堆的性质，不需要旋转。
2. 插入节点优先级 > 根结点的优先级，不满足大根堆的性质，旋转方法为把根结点旋转到根的左子树的位置，根的右子结点成为子树的根。如果右子树有左儿子，则移到原根结点的右儿子，其他保持不变。

旋转

同样，右旋和左旋完全相反。

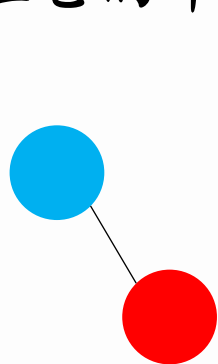


当插入节点在左子树的时候：

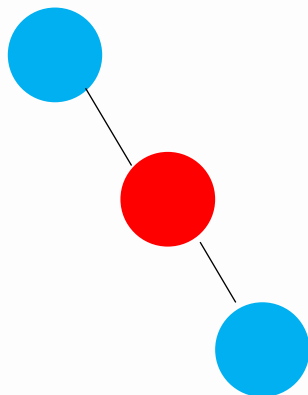
- 1、如果优先级 < 根的优先级，满足大根堆性质，不旋转
- 2、如果优先级 > 根的优先级，把左结点提到根的位置，根变为左结点的右儿子，如果该节点已经有右儿子，则原来的右儿子变为此时根的右儿子的左儿子。

删除

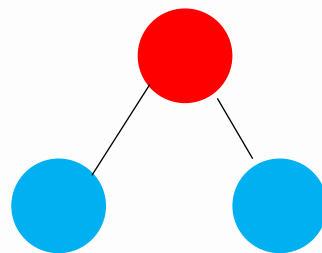
前面讲左旋和右旋的时候实际上就是讲插入操作。对于平衡树而言，如果要删除某个结点，我们该如何操作保证它的平衡性呢？



叶子结点



只有一棵子树

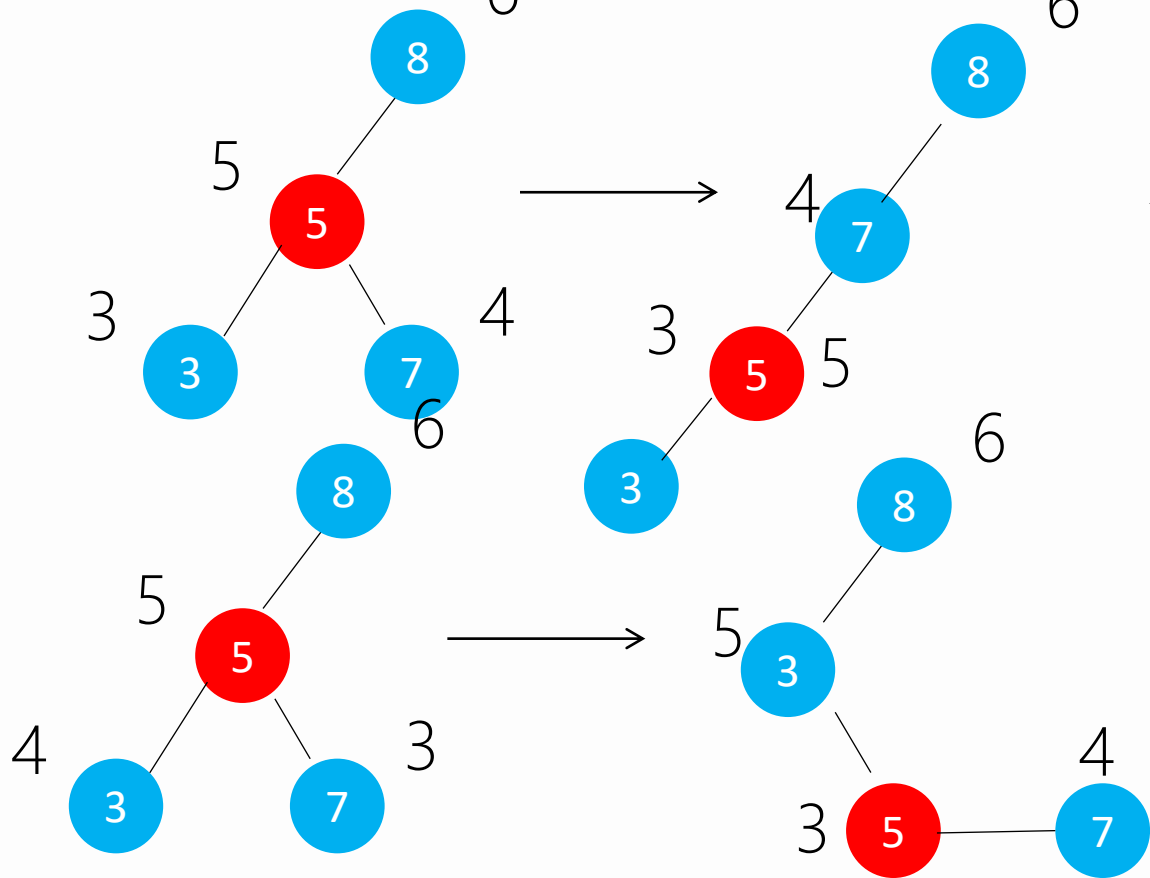


有左右子树

我们很容易发现，对于前两种情况，删除该结点很简单。如果该结点是叶子结点，直接删除不会有任何影响，如果该结点只有一个儿子，删除之后把子树提上来就可以了。

删除

如果需要删除的节点，左右子树都存在，直接删除会导致平衡树的结构发生变化，所以我们需要通过一系列的操作来保持它的平衡。我们可以考虑前面的左旋和右旋。把当前节点移动到叶子结点或者只有一个子树的位置就可以了。



经过我们模拟之后我们会发现：
左子树优先级 < 右子树：
右子树左旋。
左子树优先级 > 右子树：
左子树右旋。

注意：如果是建立的小根堆就恰好相反。